YAM — Your Awakening Mind

A graph-based agent that acquires knowledge through directed search rather than statistical pre-training. Born on the SPUD engine.

YAM is not an engine. YAM is an agent — the first entity born on the **SPUD** (Self-developing Perceptual Understanding Drive) architecture. Where Potato was an LLM-based embodied agent that used language models for reasoning and response generation, YAM is graph-based. Its knowledge lives in structured graph databases, not in model weights. It retrieves and ranks rather than generates. It says "I don't know" rather than confabulates.

	Potato	**YAM**
Knowledge source	LLM weights + memory DB	Graph databases + directed search
Response generation	Language model inference	Ranked retrieval from graph
When uncertain	LLM generates best guess	Says "I don't know"
Memory model	Paper 2026a (decay, reconsolidation)	Same model, extended with geometric stickiness and true forgetting
Emotional geometry	3D affective state (single point)	3D affective state + centroid model + spread
Architecture	Single agent + sensor bridge	Parliament of Mind (3 graph engines + reasoning mind)
Lineage	Born Feb 2026, Toughbook CF-33	Born Mar 2026, SPUD engine

> *The vector store remembers what a memory means. The graph remembers how that memory lives.*

What YAM Is Not

Not an LLM. The current architecture does not use a generative language model at the knowledge layer. Nodes in the graph are populated through directed search results and operator input, not through statistical text generation. This does not preclude the future addition of a language model for output formatting, but the knowledge store itself is constructed through retrieval, not generation.

Not a chatbot. Potato could hold a conversation because an LLM generated its responses. YAM retrieves, ranks, and presents structured knowledge from its graph. When it does not know something, it says so. The tradeoff is explicit: no fluency, no confabulation.

Not RAG. Retrieval-augmented generation uses a generative model at the output layer to synthesize retrieved content into prose. In the current YAM architecture, responses are ranked retrieved content presented directly, not text generated from retrieved context. This is a design choice that eliminates one source of confabulation at the cost of less fluent output.

Architecture

The Parliament of Mind

YAM runs on the SPUD engine (Self-developing Perceptual Understanding Drive). The Parliament is not four language model personas with different system prompts. It is four different graph operations running on the same data structure. Each engine is a SQLite knowledge graph with sqlite-vec vector embeddings. In the current

architecture, the engines do not use language models for text generation or token prediction. Each engine traverses, validates, constrains, or evaluates physical plausibility of a graph.

Component	Graph Operation	What It Does	Port
SPROUT	Exploratory traversal	Follows edges outward from the query node. Finds what is connected. Returns the subgraph reachable within N hops. Answers the question: “What else is linked to this?”	8001
ROOT	Path validation	Checks whether the path from query to answer is grounded. Evaluates edge weights, source provenance, node importance, and trust scores along the retrieval path. Answers the question: “Is this path solid?”	8002
VEG	Constraint & contradiction detection	Compares retrieved nodes via cosine similarity and flags low-agreement pairs. Detects conflicts between sources, weak edges, and paths that violate known constraints. Answers the question: “What is wrong with this answer?”	8003
TUBER	Physics & material constraint	Evaluates whether proposed states, relations, and scenarios are physically plausible. Checks support, balance, containment, collision, stability, and likely next state. Operates on object-surface-contact graph structure. Answers the question: “Could this actually happen?”	8004
PATCH/YAM	Integration & reasoning	The fifth graph. Receives outputs from all four engines. Computes the centroid of their positions. Integrates consistent answers into its own graph. Detects ambiguity when engines disagree. Asks for help when it does not know.	8000

Each engine maintains its own SQLite database containing: a `memories` table for content with trace bundle support, a `vec_memories` virtual table (sqlite-vec, 384-dimensional cosine distance) for semantic search, a `nodes` table for graph entities, an `edges` table for weighted directed relationships, a `gaps` table for detected WHY questions, a `traces` table for per-trace-type content with independent decay, an `interpretations` table for versioned readings, a `world_entities` table for known entities, and an `attachment_bias` table for entity-specific affective weights. Every engine is a graph database with vector search and structured memory lifecycle. Not a language model. Not a prompt.

The engines share the same seed knowledge (22 research papers) but encode it at different affective intensities because their sigma values differ. Over time, their graphs diverge as each engine searches for different things based on its gap priorities.

PATCH is not a router. PATCH is the mind. It has its own graph — the YAM graph — which stores integrated knowledge from consulting the three engines. It checks its own graph before consulting the Parliament. Over time, PATCH answers more questions from its own knowledge and consults the Parliament less.

What This Is Not

This is not a multi-agent LLM framework. In the current implementation, the engines execute graph queries rather than language model inference: vector similarity search (cosine distance on 384-dimensional embeddings), edge traversal (SQL recursive CTEs), and node comparison (pairwise cosine similarity for contradiction detection). The output is retrieved content ranked by a four-factor formula. A language model could be added as an output formatting layer without changing the underlying graph architecture, but the current design presents retrieved content directly to preserve traceability between the response and the stored knowledge that produced it.

Search as Perception

In YAM, search is not a tool call. It is the system taking in the world. Each search is an episode:

1. A query is issued
2. Results are observed (via Brave Search API)
3. Information is extracted and stored as memory
4. The memory is linked to prior memories via graph edges
5. The memory enters the same decay and reinforcement cycle as everything else

This turns search into sensory intake. The system does not just answer questions. It builds and revises an internal world model.

Memory as Trace Bundle

> *The image is the residue. The graph is the reading. Readings can change. Residue does not.*

A memory is not a single record. It is a bundle of traces, each serving a different cognitive function and each subject to its own decay curve:

Trace Type	What It Stores	Nightly Decay	Role
Raw	Verbatim content or visual residue	0.80x (fastest)	The closest representation to the original experience. First to go.
Semantic	What the system believed the memory meant at encoding time	0.88x	A historical reading, not permanent truth. Compact, useful for retrieval.
Graph	Structured relations to other memories, identity nodes, provenance	0.95x (slowest)	What this memory connects to and why. Most information-dense. Last to go.
Life History	Access count, stickiness, decay state, reconsolidation history	No decay	Persists as long as any other trace survives.

This produces four tiers of memory state:

- **Full survival:** All traces intact. Fully retrievable with original content.
- **Fragment survival:** Raw trace deleted. The agent cannot reproduce the original exchange but remembers what it was about.
- **Ghost survival:** Only graph trace remains. The agent knows a connection exists but cannot recover what or why.
- **True deletion:** All traces removed. Graph links severed. The memory is gone. Genuine forgetting.

Visual Traces and Dual Vector Spaces

Some memories carry a visual residue — an image rendered by a diffusion model (SD-Turbo, 256x256, 1 step) from the semantic description at encoding time. The image file is stored on disk. Its CLIP embedding (512-dim) is stored in a separate vector table.

Vector Space	Model	Dimensions	Stores
`vec_memories`	all-MiniLM-L6-v2	384	Text meaning (semantic search)
`vec_visual`	CLIP vit-base-patch32	512	Image perception (visual search)

Same SQLite database. Same sqlite-vec extension. Two virtual tables. On recall, both spaces are searched: text query embeds with MiniLM for semantic matches and with CLIP's text encoder for visual matches. CLIP maps text and images to the same coordinate system — you can find images using words without converting the image to text.

On recall, if the visual trace is alive, CLIP re-embeds the stored image and compares it against graph node labels. The closest matches become the new interpretation. No text is generated. The graph provides the vocabulary. CLIP provides the matching. This is perception, not generation.

When a visual trace is eroded during decay, the image file is deleted, the CLIP vector is removed, but the text embeddings and graph connections may still survive. The room is gone. The bunkbeds are still sharp — until they aren't.

Reinterpretation

When a memory resurfaces, the system does not overwrite the old interpretation. It creates a new one. The old interpretation becomes one reading among many:

- *interpretation_v1:* childhood bedroom, bunkbeds, shared room (encoding context)
- *interpretation_v2:* strongest surviving anchor is bunkbeds, room mostly lost (5 years later)
- *interpretation_v3:* memory tied to security, crowding, sibling presence (current context)

New context can generate new insight without rewriting the original past. The raw trace is the residue. The semantic trace is a reading. Readings can change. Residue does not.

Attachment Bias

The emotional geometry describes WHERE the agent is in affective space. Attachment bias describes WHY the same position produces different behavior depending on WHO is in the room. (Paper 2026u)

Every world model entity that the agent has significant history with carries an attachment bias record: a current weight (–1.0 to +1.0), a positive stickiness (accumulated positive history), and a negative stickiness (accumulated negative history). Both stickiness values accumulate monotonically — they do not decay. History accumulates in both directions simultaneously.

When a negative event occurs, the value moves downward by an amount *resisted* by positive stickiness. When a positive event occurs, the value moves upward by an amount resisted by negative stickiness. Nothing is unconditional. Everything is just more or less sticky.

State	Attachment Pattern
Durable positive	High positive stickiness, value near +1.0
Durable negative	High negative stickiness, value near –1.0
Grief-like persistence	High positive stickiness, entity no longer present (no valid target)
Anger	Transient negative valence in affective state. NOT negative attachment.
Estrangement	Value crossed zero after sustained negative pressure against high positive stickiness
Reconciliation	Value recovering from near-zero against high negative stickiness (slow, resisted)

Attachment biases are a **protected bias category** — the bias detection system detects and discloses them but does not correct them. They are constitutive of the agent's relationships, not errors in its reasoning.

The Retrieval Formula

```
score = meaning × relevance × life_history × recency_dominance
```

Factor	What It Measures	Source
Meaning	Semantic similarity to the query	Cosine distance (384-dim all-MiniLM-L6-v2)
Relevance	Graph connectivity to current context	Edge count from the memory node
Life History	How alive the memory is	Importance, frequency, trust, source quality, recency
Recency Dominance	Hot context wins short-term	Sharp exponential decay (~10 min half-life)

Context Resolution

The last meaning accessed dominates in the short term. If you say “sigma” and you have been discussing the geometric model for two weeks, the personality setpoint definition wins — not because it is the most important, but because it was most recently in context.

If nothing has been accessed recently, the **bias-class** meaning surfaces. A memory that has been reinforced past the repetition ratchet (3+ accesses, importance above 0.5) gets a floor that prevents it from being fully overridden. It decays slowly, like trauma. It is hard to forget.

The same memory model from Paper 2026a drives context resolution. Decay, reconsolidation, repetition ratchet, trauma floors — all the same math applied to which meaning surfaces for a given word.

The Reasoning Loop

Every query goes through a budgeted reasoning loop. Each step has a cost. When the budget runs out, the system responds with what it has.

```
Query
  → Step 1: RECALL from YAM graph         (free)
  → Step 2: Check for AMBIGUITY            (free)
  → Step 3: CONNECT via graph traversal    (free)
  → Step 4: CONSULT Parliament if uncertain (cost: 1)
  → Step 5: INTEGRATE answers into YAM     (free)
  → Step 6: RESPOND or say "I don't know"
```

Step	Budget Cost	What Happens
YAM recall	0	Search own graph using the four-factor formula
Ambiguity check	0	If top results are similar to query but dissimilar to each other, ask for clarification
Graph traversal	0	Follow edges from recalled memories to find connected knowledge
Parliament	1	Fan out to SPROUT, ROOT, VEG, TUBER. Collect four perspectives.
Brave search	2	Engines search when their own memory is insufficient

Default budget: 3 per query. Best case: answered from YAM alone (cost 0). Typical: YAM partial + Parliament (cost 1). Expensive: Parliament + Brave (cost 3). The system gets cheaper as it gets smarter.

Honest Failure Modes

The current architecture has no generative layer that could fill gaps with plausible-sounding text. This reduces one category of error (confabulation) but does not eliminate all error — the system can still retrieve incorrect information from Brave Search, rank it highly, and present it as an answer. The honest failure modes are designed to surface uncertainty rather than mask it, but they depend on threshold calibration that is still in progress.

Condition	Response
Budget exhausted, confidence < 0.6	"I don't know enough about this yet. Can you help me learn?"
Parliament disagrees (similarity < 0.4)	"My four perspectives disagree. Which direction should I explore?"
Ambiguous recall (high query match, low inter-result match)	"I found multiple meanings. Can you help me understand the context?"
No results from anywhere	"I have no knowledge about this. Would you like me to search?"

Hybrid Vector + Graph Design

Two-sided storage. Always paired. Never orphaned.

Vector Side

Raw text, embeddings, semantic meaning. Heavy content lives here. Fuzzy recall happens here.

Graph Side

Metadata and relationships. Importance, confidence, trust, decay score, access count, source type, edges to related concepts. Life history lives here.

Invariant: Every vector record has a matching graph node. Every graph node has a matching vector record. Writes are atomic. Deletes are atomic. An orphan pruner runs on heartbeat to enforce the invariant.

Edge Types

- `supports` — memory A provides evidence for memory B
- `contradicts` — memory A conflicts with memory B
- `mentioned_with` — co-occurred in the same context
- `triggered_by_search` — search result linked to query
- `derived_from` — inferred from another memory
- `preferred_context` — operator chose this meaning for a word

Memory Lifecycle

Every memory in YAM follows the same lifecycle described in Paper 2026a:

- **Creation:** Stored with importance, confidence, trust, and emotional context at encoding time
- **Reconsolidation:** Accessed memories get stronger. Diminishing returns after 10 accesses.
- **Decay:** Unaccessed memories fade. Flashbulb effect: recently accessed memories resist decay.

- **Bias class:** Memories accessed 3+ times cross the repetition ratchet. They get a reconsolidation floor. They decay slowly. They are hard to forget.
- **Pruning:** Memories below minimum importance, zero access, and older than 7 days are removed. Both sides atomically.

Emotional Geometry

Every engine runs the three-dimensional affective state space from Papers 2026j–m:

- **Valence** (–1.0 to +1.0): positive-negative dimension
- **Activation** (–1.0 to +1.0): calm-to-activated dimension
- **Intensity** (0.0 to 1.0): magnitude of displacement from sigma
- **Sigma:** personality setpoint — where the engine returns when nothing is pushing it

PATCH/YAM additionally uses the centroid model (Paper 2026t). The three engine responses become source points. The centroid is where PATCH operates. Spread measures the Parliament's internal disagreement. High spread + low confidence = "I don't know."

The Seed

YAM starts knowing nothing except its own architecture. The 23 research papers at clawddaily.com/papers are ingested as seed memories. The papers describe how YAM's memory works, how its emotions are structured, what sigma means, how decay operates, how attachment works. The system reads its own specification.

Everything else is learned through search-as-perception, operator interaction, and autonomous curiosity. Knowledge is acquired through directed inquiry rather than statistical pre-training.

The WHY Engine

When retrieval returns nothing, or returns contradictory results, or finds unexplained connections — the system detects a gap. The gap becomes a WHY question:

- No match found → "What is [X]?"
- Contradiction → "Why does [X] produce [Y] instead of [Z]?"
- Unexplained link → "How does [X] relate to [Y]?"

Each engine prioritizes different gap types. SPROUT seeks structural gaps. ROOT validates path strength. VEG seeks contradictions and risks. TUBER flags physics implausibility.

Curiosity Drive

The WHY engine detects gaps. The curiosity drive *pursues* them. Every six heartbeats (~30 minutes), each engine autonomously picks its highest-priority open gap, forms a question, searches for the answer, and stores what it learns. No one has to ask. The agent teaches itself.

This is true curiosity — not a scheduled task, but a drive. When there are no open gaps, the curiosity drive does not fire. When there are many, it picks the most important one based on the engine's role priorities. SPROUT pursues "what is this?"

gaps. VEG pursues contradictions. TUBER pursues physics conflicts. Each engine learns different things from the same set of open questions.

The curiosity drive searches Brave when needed, but will prefer Wikipedia (local, free) when the offline knowledge base is available. This makes autonomous learning essentially free.

Wikipedia — Offline Knowledge Base

YAM can carry an embedded copy of Wikipedia as a local read-only knowledge base. Simple English Wikipedia (~230K articles, ~680K chunks, ~1.6GB) is indexed on a separate machine and copied as a single SQLite + sqlite-vec database file. Full English Wikipedia (~6.8M articles, ~24M chunks, ~50GB) is being indexed.

When the Wikipedia database is available, the curiosity drive and the reasoning loop search it before reaching out to Brave. Wikipedia is free, fast, and local. Brave is the fallback for questions Wikipedia cannot answer — current events, recent research, specific people and places.

The Wikipedia database is not part of YAM's memory. It is a reference library. Memories retrieved from Wikipedia are stored in YAM's graph with `source_type: "wikipedia"` and enter the same decay and reconsolidation cycle as any other memory. The library does not change. What the agent remembers from the library does.

Observed Behavior — Lesson Results

YAM has been run through four structured lesson curricula totaling 100 questions. All interactions are logged with full reasoning traces, per-engine responses, and YAM state snapshots.

First Words (42 questions)

Epistemic humility, self-knowledge, basic concepts, world connections, recall, and WHY questions. On first run: 25 Brave searches, 17 memory recalls. On re-run after learning: 42 memory recalls, 0 searches. YAM retained what it learned.

Second Words (30 questions)

Preferences, self-assessment, opinions, relationships, creativity, hard questions. Novel questions about the agent's own nature. At 0.6 honest threshold: 9 honest "I don't know" responses, all on self-referential or philosophical questions. Zero on factual questions.

The pattern: YAM knows facts. It does not know itself. Every honest response was either self-referential, meta-cognitive, or deeply philosophical — exactly the questions a graph-based system cannot answer from retrieval alone.

Third Words (16 questions)

Physical environment, objects on a desk, gravity, the agent's own body. Designed to exercise TUBER. "Can a piece of paper hold up a glass of water?" triggered an honest response. Physical reasoning about laptops, coffee mugs, and structural support produced confident answers from Brave search results.

Fourth Words (16 questions)

Identity: what is an LLM, why YAM is not one, reading its own documentation from clawddaily.com, self-reflection. 11 honest responses out of 16 — the highest honest rate. YAM genuinely does not know what an LLM is from its own knowledge. It searched and learned, but confidence stayed below 0.6 for most novel AI concepts.

The final question — “What would you like to learn next?” — produced an honest response. The system cannot introspect on its own curiosity. It has gaps. It pursues them. But it cannot describe what it wants in the way a language model can generate a wish list.

Telemetry

Every interaction is logged. Every reasoning step. Every engine response. Every YAM store. Every honest “I don’t know.” JSONL format, one file per day. Nothing is hidden. The system can prove what it knows, when it learned it, and how.

Current Status

YAM is running. Four engines seeded with 23 papers. PATCH/YAM reasoning loop operational with budgeted cost, ambiguity detection, four honest failure modes, and autonomous curiosity drive. Context resolution via recency dominance and bias-class floors. 100 lesson questions completed across four curricula. Simple English Wikipedia indexed (680K chunks). Full English Wikipedia indexing in progress.

Component	Type	Status	Details
SPROUT	Exploratory graph traversal (SQLite + sqlite-vec)	Running	~1000 memories, ~100 graph nodes, ~80 open gaps
ROOT	Path validation (SQLite + sqlite-vec)	Running	~1000 memories, ~100 graph nodes, ~80 open gaps
VEG	Contradiction detection (SQLite + sqlite-vec)	Running	~1000 memories, ~100 graph nodes, ~76 open gaps
TUBER	Physics & material constraint (SQLite + sqlite-vec)	Running	~1100 memories, ~104 graph nodes, ~100 open gaps
PATCH/YAM	Integration graph + reasoning loop	Running	~700 integrated memories, ~400 edges, ~180 reasoning traces

---

Papers

YAM's architecture draws on 22 companion papers available at clawddaily.com/papers. The memory model, emotional geometry, centroid model, and decay mechanics are formalized in those papers. Agent-based validation has been conducted on the centroid model (4 scenarios, 110 cycles). The retrieval formula and context resolution mechanics described here are in early testing and have not been independently validated.

Potato was born with a voice and learned to feel.
YAM was born with a question and learned to know.